IndustryApp – Software Design Document

Nov 9, 2026 · @efe

Document information

This document describes the design of **UC-04 Select Offer and Reveal Contact Information**, implemented in release v0.2 of IndustryApp.

Field	Value
Course	BIL 482 Design Patterns
Deliverable	Project Assignment 2 – Software Design Document
Selected use case	UC-04 Select Offer and Reveal Contact Information
Authors	Ömer Efe Öztürk, Bahadır İhsan Herdem
Version	1.0 (describes software release v0.2)
Repository	github.com/BahadirIhsan/bil482

Traceability codes from PA1 are kept unchanged: UC-04 → FR-04.n → PAT-04.n → code comment `// [UC-04][PAT-04.n]` → test TC-04.n.k → commit `[UC-04] <type>: <description>`.

Document task matrix

#	Task in creating this document	Responsible	Reviewer
T1	Sections 1–4: overview, context, features, assumptions	Ömer Efe Öztürk	Bahadır İhsan Herdem
T2	Sections 5–6: architecture, components, diagrams	Bahadır İhsan Herdem	Ömer Efe Öztürk
T3	Section 7: ER diagram, data flow, validation rules	Bahadır İhsan Herdem	Ömer Efe Öztürk
T4	Section 8: patterns, class and sequence diagrams	Ömer Efe Öztürk	Bahadır İhsan Herdem
T5	Sections 9–11: implementation notes, UI, external interfaces	Ömer Efe Öztürk	Bahadır İhsan Herdem

#	Task in creating this document	Responsible	Reviewer
T6	Sections 12–15: performance, errors, testability, deployment	Bahadır İhsan Herdem	Ömer Efe Öztürk
T7	Sections 16–17, final review, PDF export, GitHub upload	Ömer Efe Öztürk	Bahadır İhsan Herdem

1. System Overview

IndustryApp is a reverse-auction marketplace that connects drivers after an accident with repair mechanics. The customer reports the damage, the system notifies matching mechanics, mechanics send sealed offers and the customer selects one. Contact details are revealed only at the moment of the match.

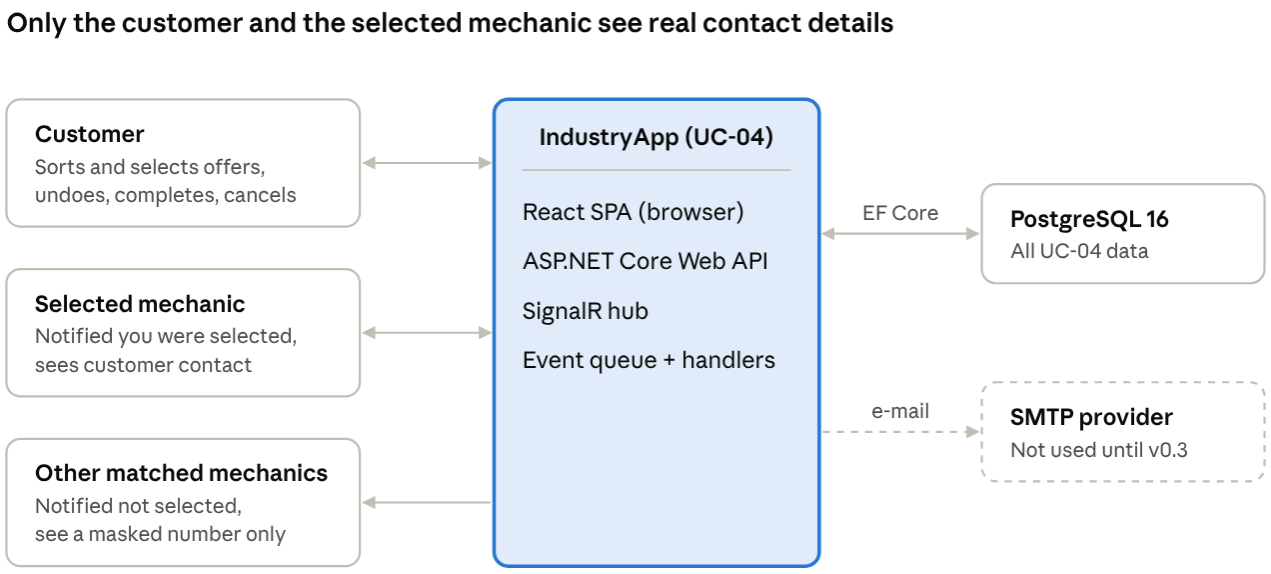
UC-04 is the closing step of this flow. The customer compares the offers, selects one, may undo the selection within 10 minutes, and then both parties see each other's contact details. Finally the customer marks the job completed or cancels the report.

Requirement	Description
FR-04.1	The customer sees offers live and sorts them by price, duration or newest.
FR-04.2	The customer selects one offer; the other offers are marked not selected.
FR-04.3	The customer can undo the selection within the configured time (default 10 minutes).
FR-04.4	Contact details are shown only to the customer and the selected mechanic.
FR-04.5	The customer marks the job completed or cancels the report.

Scope of v0.2. UC-04 runs end to end: REST endpoints, application services, domain logic, database, real-time notifications and the React screens. Login and registration (UC-01) are delivered in the same release because UC-04 needs an authenticated customer and mechanic. Report creation (UC-02) and offer submission (UC-03) follow in v0.3; until then reports and offers are inserted locally with `database/dev/uc04-dev-data.sql`.

2. System Context

UC-04 involves three actors and two external systems; only the customer acts, the mechanics are notified.



System context of UC-04 · 3 actors, 2 external systems

All actors use the React SPA over HTTPS (REST) and WebSocket (SignalR). The admin takes no part in UC-04. UC-02 and UC-03 feed UC-04 with an Open report and Pending offers; in v0.2 these come from the development data script.

3. Key Features and Functionality

UC-04 delivers seven features; each one maps to a requirement, an endpoint and a pattern.

Feature	What the user gets	FR	Endpoint (#)	Pattern
Live offer list	Offers appear without page refresh	FR-04.1	GET /api/reports/{id}/offers (27)	PAT-03.1 Observer
Offer sorting	Sort by price, fastest completion or newest	FR-04.1	GET …/offers?sort= (27)	PAT-04.4 Strategy
Price breakdown	Labor, parts, paint, warranty, other lines per offer	FR-04.1	GET …/offers (27)	–

Feature	What the user gets	FR	Endpoint (#)	Pattern
Offer selection	One click + confirm; others become NotSelected	FR-04.2	POST /api/reports/{id}/selection (29)	PAT-04.1 State, PAT-04.2 Command
Undo selection	Countdown banner, undo within 10 min	FR-04.3	DELETE /api/reports/{id}/selection (30)	PAT-04.2 Command
Contact reveal	Name, phone, e-mail for the two matched parties only	FR-04.4	GET /api/reports/{id}/contact (32)	PAT-04.3 Proxy
Complete / cancel	Close the job or cancel the report	FR-04.5	POST …/complete (20), …/cancel (19)	PAT-04.1 State

Main flow

1. The customer opens the report detail screen and sorts the offers.
2. The customer opens a price breakdown, presses "Select this offer" and confirms.
3. `SelectOfferCommand` runs: report → OfferSelected, chosen offer → Accepted, others → NotSelected; a snapshot is saved in the status history.
4. `OfferSelectedEvent` is published; the winner is notified "you were selected", the others "another offer was selected".
5. Both matched parties see each other's contact details; everyone else sees a masked number.
6. During the undo window the customer sees a countdown.
7. When the work is done, the customer marks the report completed.

Alternative flows and exceptions

Code	Situation	Result
2a	User is not the report owner	403 `NOT_REPORT_OWNER`
3a	Two selection requests at the same time	Second request 409: `CONCURRENCY_CONFLICT` (xmin), `UNIQUE_VIOLATION` (partial unique index) or `REPORT_INVALID_STATE`

Code	Situation	Result
3b	Offer was withdrawn meanwhile	409 `OFFER_NOT_AVAILABLE`
5a	User unrelated to the report opens the contact screen	403 `NOT_MATCHED`; a matched but not selected mechanic gets a masked number
5b	Customer opens the contact screen before selecting	409 `NO_ACTIVE_SELECTION`
6a	Undo within the window	Command rebuilt from snapshot; report Open, offers Pending, `OfferSelectionUndoneEvent`
6b	Undo after the window	409 `UNDO_WINDOW_EXPIRED`
7a	Customer cancels the report	Report Cancelled; Pending and Accepted offers Closed, NotSelected offers keep their status

4. Assumptions and Dependencies

Assumptions

- **A-1** The customer owns an Open report with at least one Pending offer (precondition of UC-04). In v0.2 this data comes from `database/dev/uc04-dev-data.sql`.
- **A-2** Users register and log in through the v0.2 screens; mechanics are approved locally with one SQL update until the admin screen (YA-01) exists.
- **A-3** A report has exactly one customer and at most one accepted offer.
- **A-4** The undo window, token lifetimes and other limits come from `appsettings.json` (`Offers:SelectionUndoWindowMinutes` = 10).
- **A-5** The system runs as one API process on one server; the event queue is in memory.
- **A-6** The domain never reads the system clock; time is passed in as `DateTimeOffset now` from `IDateTimeProvider`.
- **A-7** The UI is Turkish; dates are `dd/MM/yyyy HH:mm`, empty values are shown as `-`.

Dependencies

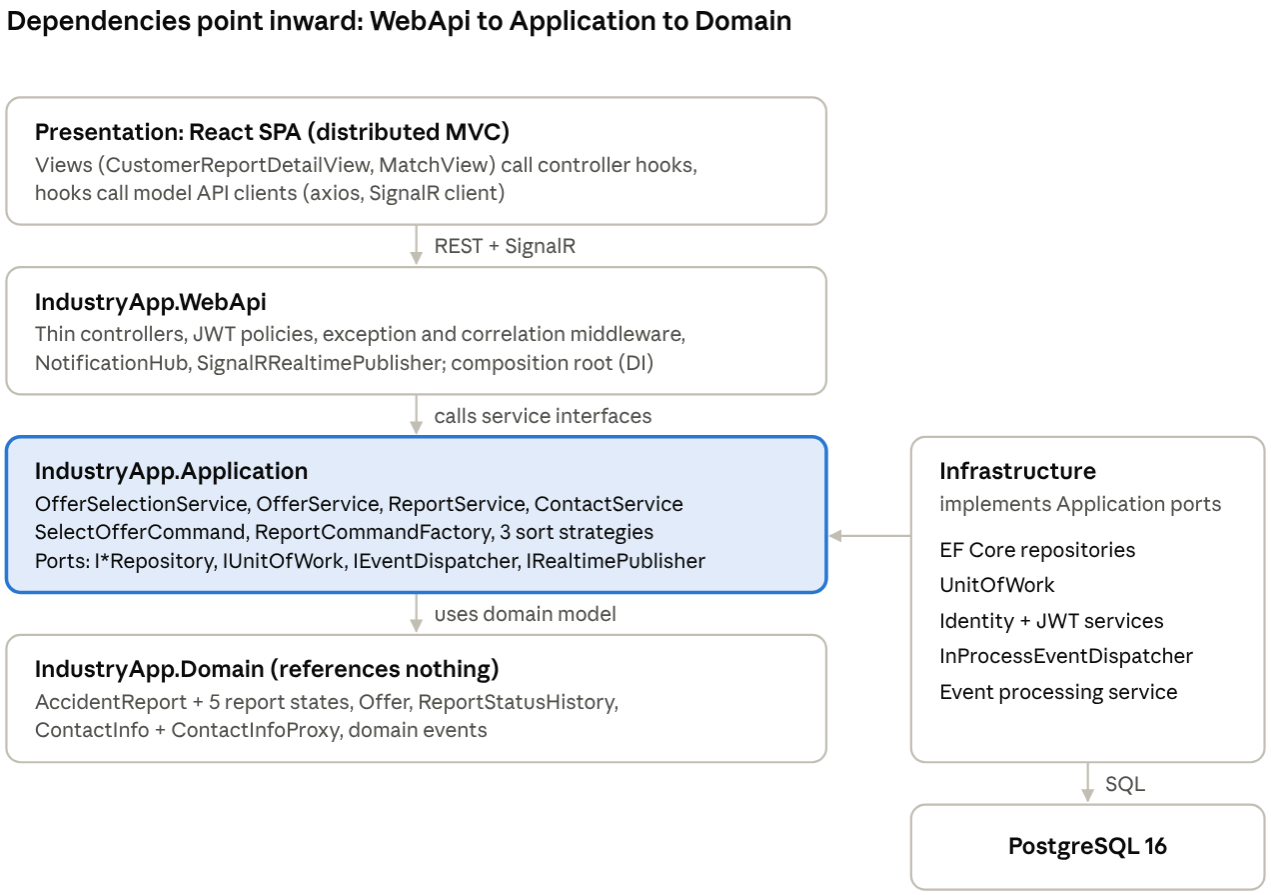
Area	Dependency	Version
Runtime	.NET SDK (pinned in `global.json`)	8.0.4xx
Web API	ASP.NET Core Web API, SignalR, JwtBearer	8.x

Area	Dependency	Version
Identity	ASP.NET Core Identity	8.x
Persistence	EF Core + Npgsql, EFCore.NamingConventions	8.x
Database	PostgreSQL (Docker image `postgres:16-alpine`)	16
Application	FluentValidation, Scrutor, Microsoft.Extensions.Options	latest stable
Logging / docs	Serilog.AspNetCore, Swashbuckle	latest stable
Frontend	React, TypeScript, Vite, MUI, TanStack Query, axios, React Hook Form + Zod, @microsoft/signalr	React 18, Node ⩾ 20
Tests	xUnit, NSubstitute, FluentAssertions, Testcontainers.PostgreSql	latest stable
Tooling	Docker Compose, dotnet-ef (local tool)	–

5. Architectural Design

IndustryApp is a modular monolith built in Clean Architecture layers; modules talk through an in-process publish–subscribe event bus.

5.1 System architecture diagram (high-level)



High-level architecture · 4 backend layers, 1 SPA, 1 database

Infrastructure implements the interfaces (ports) that Application defines, so Application never references EF Core, Identity or SignalR. WebApi references Infrastructure only to register it in `Program.cs`.

5.2 Architectural patterns and styles

Style	Where	Role in UC-04
Layered / Clean Architecture	Backend: Domain, Application, Infrastructure, WebApi	Selection rules live in Domain and Application, free of frameworks
Modular monolith	Modules Reports, Offers, Contacts, Notifications, Accounts	One process, one transaction for report + offers + history
Publish–subscribe (event-driven)	`IEventDispatcher` + `IEventHandler<T>`	`OfferSelectedEvent` triggers notifications without the selection service knowing them

Style	Where	Role in UC-04
Distributed MVC	React: View → controller hook → model API	`CustomerReportDetailView` → `useSelectOffer` → `selectionApi.select`
REST + server push	REST for actions, SignalR for updates	Every change goes through REST; the hub only pushes

5.3 Rationale for architectural decisions

No	Decision	Rationale
K-01	Modular monolith, Clean Architecture	Two-person team; a selection changes report, offers and history in one transaction; modules can be split later because they talk through events
K-02	In-process event bus	The selection service does not know who must be notified; a new listener changes no existing code
K-05	PostgreSQL 16	Partial unique index enforces "one accepted offer per report" in the database
K-06	Sealed offers	A mechanic never sees other offers; the response models have no field for them
K-11	Identity + JWT access and refresh tokens	Web and future mobile client use the same mechanism
K-12	Every service behind an `I`-interface	Layers depend on contracts; services are replaced by fakes in tests
K-13	One thin controller per resource	Controllers never call each other; shared work sits in an application service

6. Component Design

UC-04 is spread over five modules; each module has the same folder name in every layer.

6.1 Subsystems and modules

Module	Backend components	Frontend components
Reports	`ReportsController`, `IReportService`, `AccidentReport` + states	`CustomerReportDetailView`, `useCustomerReport`
Offers	`OffersController`, `IOfferService`, sort strategies, `Offer`	`OfferList`, `SortSelect`, `PriceBreakdown`, `useReportOffers`
Selection	`OfferSelectionController`, `IOfferSelectionService`, `SelectOfferCommand`, `ReportCommandFactory`	`SelectionBanner`, `useSelectOffer`, `useUndoSelection`
Contacts	`ContactsController`, `IContactService`, `ContactInfoProxy`	`MatchView`, `useMatchContact`
Notifications	`NotificationsController`, `INotificationService`, UC-04 event handlers, `NotificationHub`	`NotificationBell`, `RealtimeProvider`, `useNotifications`

6.2 Responsibilities of each component

Component	Responsibility	Does not
Controllers	Bind request, call one service method, map `Result` to HTTP	Contain business logic or call other controllers
`OfferSelectionService`	Load report with offers, check ownership, run or undo the command, write history, save	Decide if a transition is allowed (State does)
`OfferService`	Check ownership, pick sort strategy, list Pending/Accepted/NotSelected offers	Sort in memory
`ContactService`	Find the other party, wrap real contact in `ContactInfoProxy`	Return a real phone number directly
`ReportService`	Report detail, complete and cancel through State	Change status fields directly

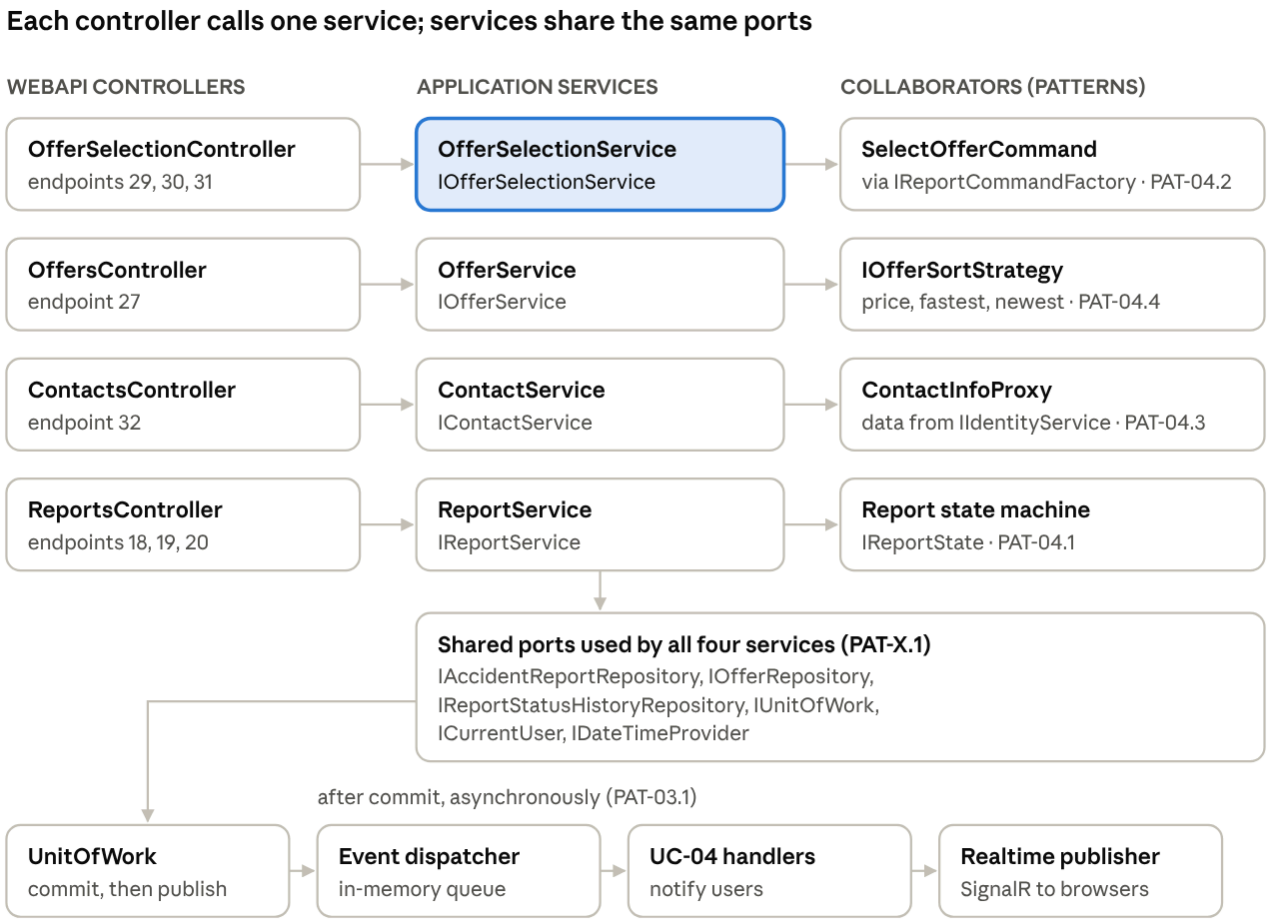
Component	Responsibility	Does not
`AccidentReport` + states	Allow or reject each operation, raise domain events	Access the database
`UnitOfWork`	One transaction, translate conflicts to 409, hand events to dispatcher after commit	Publish events before commit
Event handlers	Create notifications, push SignalR messages	Change report or offer state

6.3 Interfaces between components

Interface	Main members
`IOfferSelectionService`	`SelectAsync(ISelectOfferRequest, ct)` → `SelectionResponse`; `UndoAsync(IUndoSelectionRequest, ct)` and `GetAsync(reportId, ct)` → `SelectionStatusResponse`
`IOfferService`	`ListForReportAsync(IReportOffersQuery, ct)` → `OfferListItemResponse[]`
`IReportService`	`GetForCustomerAsync(reportId, ct)` → `ReportDetailResponse`; `CompleteAsync(reportId, ct)`; `CancelAsync(reportId, ct)`
`IContactService`	`GetMatchContactAsync(reportId, ct)` → `MatchContactResponse`
`IReportCommandFactory`	`CreateSelectOffer(report, offer, actorId, now)`, `Restore(report, historyEntry, offers, actorId, now)`
`IReportCommand`	`Name`, `ActorId`, `Execute()`, `Undo()`, `CreateSnapshot()`
`IOfferSortStrategy`	`Key`, `Apply(IQueryable<Offer>)`
`IAccidentReportRepository`	`GetWithOffersAsync(id, ct)`, `GetWithDetailsAsync(id, ct)`
`IReportStatusHistoryRepository`	`AddAsync(entry, ct)`, `GetLatestAsync(reportId, commandName, ct)`

Interface	Main members
`IUnitOfWork`	`SaveChangesAsync(ct)`
`IEventDispatcher` / `IEventHandler<T>`	`EnqueueAsync(events, ct)` / `HandleAsync(event, ct)`
`IIdentityService`	`GetContactAsync(userId, ct)` → `IUserContact` (real contact, only ever wrapped by the proxy)
`IRealtimePublisher`	`PublishToUserAsync(userId, eventName, payload, ct)`

6.4 Component diagram



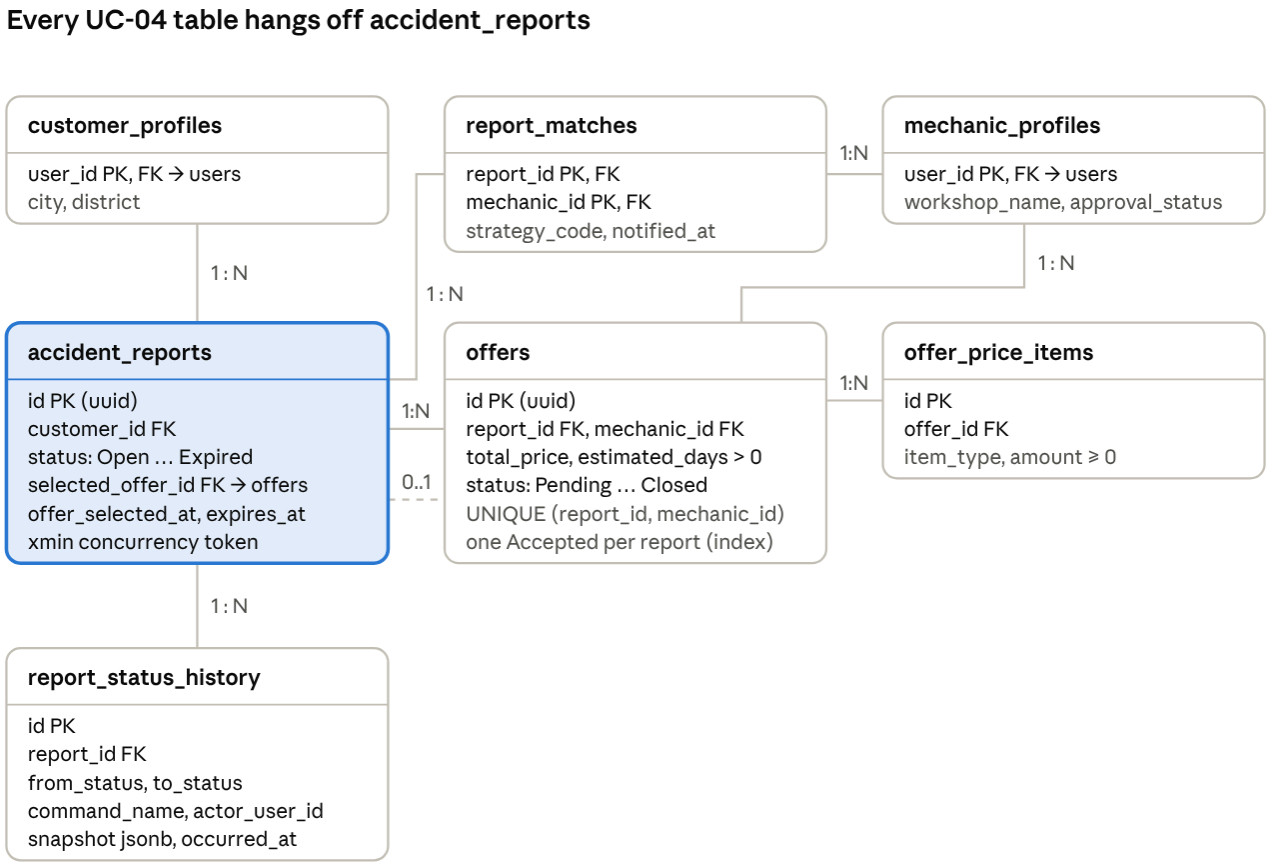
Component diagram of UC-04 · 4 controllers, 4 services, event flow

The highlighted service carries the main flow. All four services reach the database only through the shared ports; notifications leave the request path after commit.

7. Data Design

UC-04 reads and writes seven of the 21 application tables; the database itself guarantees "one accepted offer per report".

7.1 Data model / ER diagram



ER diagram of UC-04 tables · 7 tables

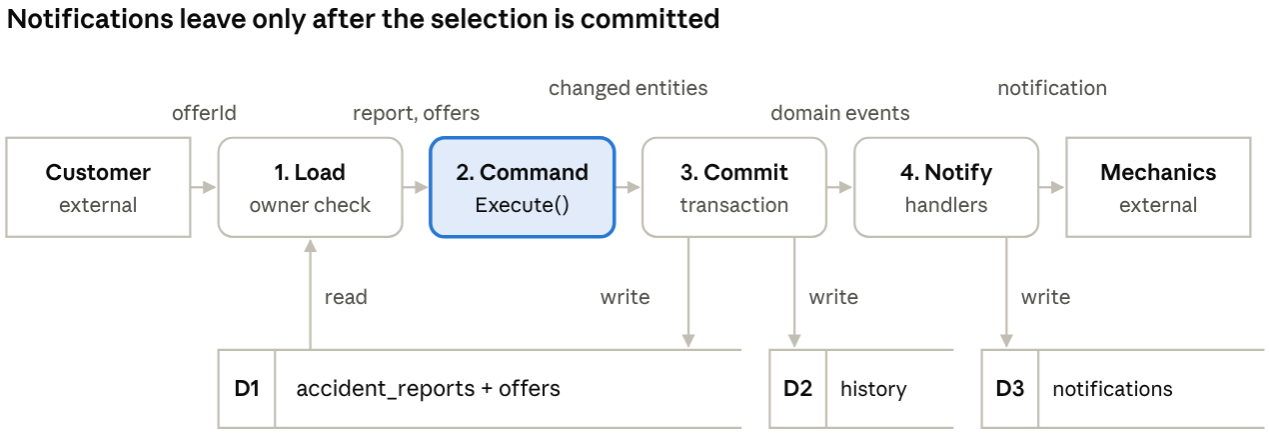
`users` (ASP.NET Identity) holds name, e-mail and phone; `ContactInfoProxy` reads them through `IIdentityService`. `notifications` and `notification_deliveries` store what UC-04 handlers send.

7.2 Data storage

- PostgreSQL 16; schema generated by EF Core migrations, snake_case plural table names.
- Keys visible to clients are `uuid`; lookup tables use integer identity keys.
- Enums are stored as text with CHECK constraints (`status IN ('Open','OfferSelected',…)`).
- `CHECK (status <> 'OfferSelected' OR selected_offer_id IS NOT NULL)` keeps the report consistent.

- Partial unique index `ux_offers_one_accepted_per_report ON offers(report_id) WHERE status = 'Accepted'` is the last line of defence against double selection.
- The PostgreSQL `xmin` column is the optimistic concurrency token of `accident_reports`.
- `report_status_history.snapshot` (jsonb) stores the Command snapshot, e.g. `{"selectedOfferId":"…","notSelectedOfferIds":["…"]}`.

7.3 Data flow diagram



Level-1 data flow of offer selection · 4 processes, 3 data stores

Undo follows the same path: P1 also reads the latest `SelectOfferCommand` row from D2, P2 runs `Undo()`, P3 writes an `UndoSelectOffer` row.

7.4 Data validation rules

Rule	Enforced in	Error
`reportId` is a GUID; `offerId` is not empty	Route constraint `{reportId:guid}`, `OfferSelectionService`	400 `VALIDATION_FAILED`
`sort` is `price`, `fastest` or `newest`; anything else falls back to `price`	`OfferService`	–
Caller owns the report	`OfferSelectionService`, `ReportService`	403 `NOT_REPORT_OWNER`
Offer belongs to the report and is Pending	`OfferSelectionService`, `AccidentReport.SelectOffer`	404 `OFFER_NOT_FOUND` / 409 `OFFER_NOT_AVAILABLE`
Report is Open for selection; OfferSelected for undo and complete	State classes	409 `REPORT_INVALID_STATE`

Rule	Enforced in	Error
`now − OfferSelectedAt ≤ SelectionUndoWindowMinutes`	`OfferSelectedState.UndoSelection`	409 `UNDO_WINDOW_EXPIRED`
At most one Accepted offer per report	Partial unique index + xmin token	409 `UNIQUE_VIOLATION` / `CONCURRENCY_CONFLICT`
Contact requested by the customer or a mechanic matched with the report	`ContactService`, `ContactInfoProxy`	403 `NOT_MATCHED` or masked data
`total_price > 0`, `estimated_days > 0`, `amount ≥ 0`	DB CHECK + `Offer.Create`	409

8. Design Patterns

UC-04 applies the four patterns mapped to it in PA1 (PAT-04.1–04.4) and reuses three system-wide ones; every pattern class carries the PA2 comment template.

8.1 Applied design patterns

Code	Pattern	Main classes	Measurable effect	Verifying test
PAT-04.1	State	`IReportState`, `ReportStateBase`, 5 states, `ReportStateFactory`	100% of invalid transitions rejected	TC-04.2.1, TC-04.2.2
PAT-04.2	Command	`IReportCommand`, `SelectOfferCommand`, `IReportCommandFactory`	State after undo = state before selection	TC-04.3.1, TC-04.3.2
PAT-04.3	Proxy (protection)	`IContactInfo`, `ContactInfo`, `ContactInfoProxy`	0 real numbers in unauthorized responses	TC-04.4.1, TC-04.4.2
PAT-04.4	Strategy (sorting)	`IOfferSortStrategy`, `PriceAscendingSort`, `FastestCompletionSort`, `NewestFirstSort`	New sort order = 1 new class, 0 changed	TC-04.1.2–4

Code	Pattern	Main classes	Measurable effect	Verifying test
PAT-03.1	Observer / Publish–Subscribe (reused)	`IEventDispatcher`, `IEventHandler<T>`, `InProcessEventDispatcher`	Selection response time independent of notification count	E2E-04
PAT-X.1	Repository + Unit of Work	`I*Repository`, `IUnitOfWork`	100% of unit tests run without a database	all unit tests
PAT-X.2	Singleton via DI	`IDateTimeProvider`, event queue	0 static `Instance` fields	code review
PAT-01.1	Factory Method (supporting, UC-01)	`IUserProfileFactory`, `CustomerProfileFactory`, `MechanicProfileFactory`	New role = 1 new factory, 0 changed classes	registration tests

8.2 Context and justification

PAT-04.1 State. A report may only be selected while Open and only undone while OfferSelected. Without State these checks would be `if (status == …)` blocks spread across services, and one missed check would allow an illegal transition. `AccidentReport` delegates every operation to `ReportStateFactory.For(Status)`; `ReportStateBase` denies everything by default and each state overrides only what it allows. A new status is one new class.

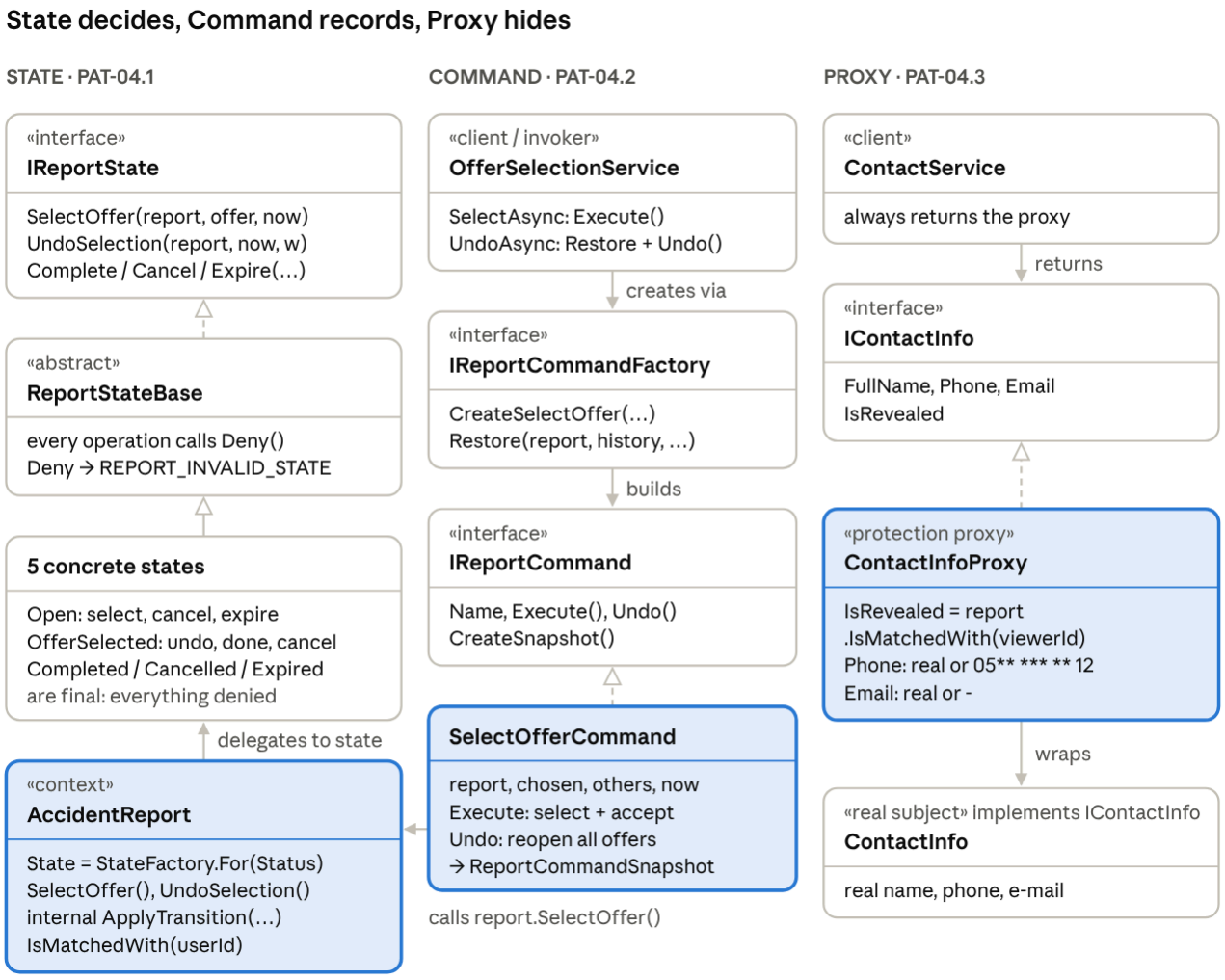
PAT-04.2 Command. Selecting changes the report and several offers together and must be reversible for 10 minutes. `SelectOfferCommand` keeps `Execute()` and `Undo()` symmetric in one object. Because HTTP is stateless, `CreateSnapshot()` persists the affected offer ids in `report_status_history`; `ReportCommandFactory.Restore` rebuilds the command for undo. The history also serves as the audit trail. The alternative, a separate "undo service", would duplicate the selection logic in reverse.

PAT-04.3 Protection Proxy. The decision to show a phone number must be made in exactly one place. `ContactInfoProxy` implements the same `IContactInfo` as the real object and returns real data only when `report.IsMatchedWith(viewerId)`; otherwise `05** *** ** 12` and `-`. `ContactService` always returns the proxy, so every new screen is protected automatically.

PAT-04.4 Strategy (sorting). The customer sorts by price, fastest completion or newest, and more orders (e.g. rating) will follow. `OfferService` picks the strategy whose `Key` matches the request, defaulting to price. Each strategy adds `ORDER BY` to the `IQueryable`, so sorting runs in SQL and the controller has no switch statement.

Reused patterns. `OfferSelectedEvent` and `OfferSelectionUndoneEvent` go through the same publish–subscribe bus as UC-03, so the selection service does not know who is notified. Repository + Unit of Work saves report, offers and history in one transaction and publishes events only after commit. The clock and the event queue are DI singletons, replaceable by fakes in tests.

8.3 Class diagram



Class diagram of State, Command and Proxy in UC-04

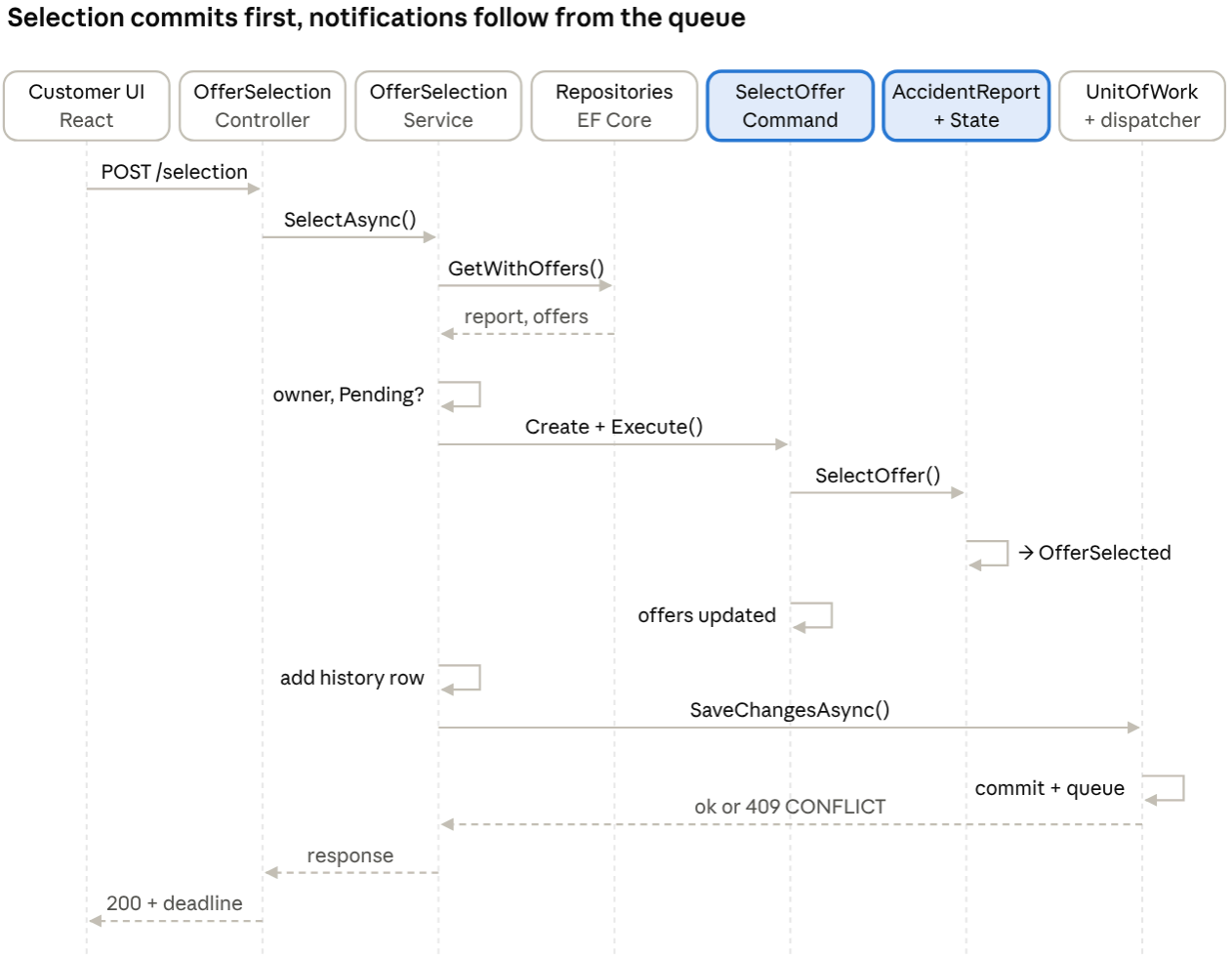
Solid arrows are associations, dashed arrows with hollow heads are interface realizations, the solid hollow head is inheritance.

Example of the comment template in the code:

```
// [UC-04][PAT-04.2] Command
// Niyet   : Teklif seçimini yürütülebilir ve geri alınabilir tek bir nesne
olarak paketlemek.
// Neden   : Seçim aynı anda raporu ve birden fazla teklifi değiştirir ve 10
dakika içinde birebir geri
//           alınabilmelidir. Alternatif olan ayrı bir "geri alma servisi",
seçim mantığını tersten ikinci
//           kez yazardı ve iki kopya zamanla birbirinden ayrışırdı.
// Kazanım : Execute ve Undo simetrik olarak tek sınıfta durur;
CreateSnapshot sayesinde komut veritabanına
//           yazılıp sonraki istekte yeniden kurulabilir ve her seçim bir
denetim kaydı bırakır.
// Metrik  : Undo sonrası durum = seçimden önceki durum; süre dışında geri
alma %100 reddedilir
//           (Undo_WithinWindow_RestoresPendingOffers TC-04.3.1,
Undo_AfterWindow_Throws TC-04.3.2).
public sealed class SelectOfferCommand : IReportCommand { … }
```

The template lines are Intent (Niyet), Reason (Neden), Benefit (Kazanım) and Metric (Metrik) with the verifying test; comments in the code are Turkish like the rest of the code base.

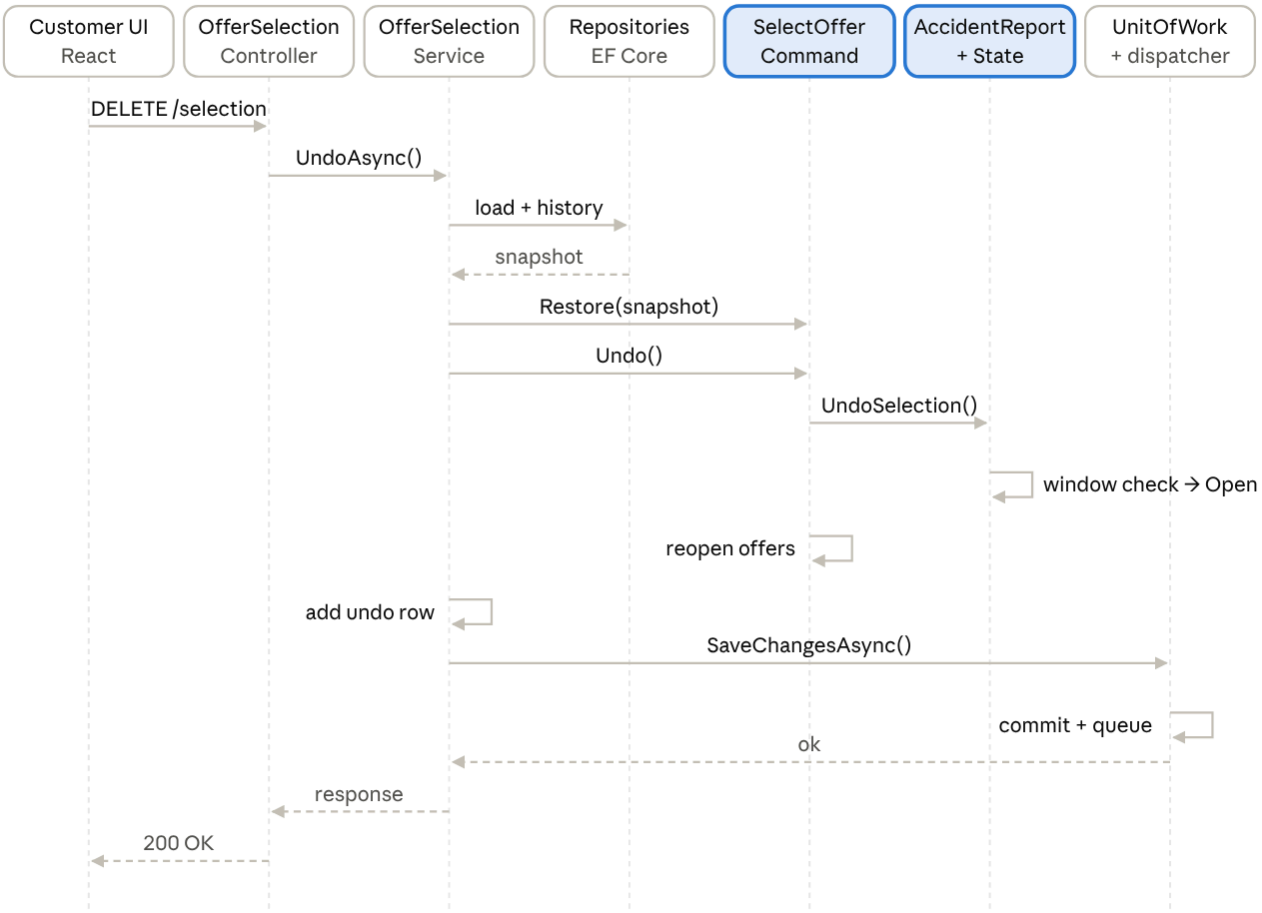
8.4 Sequence diagrams



Sequence diagram · select an offer (FR-04.2)

If a second request selects at the same time, `SaveChangesAsync` fails on the partial unique index or the `xmin` token and returns 409. After commit the dispatcher runs `OfferSelectedNotificationHandler` and `ReportStatusRealtimeHandler` in the background.

Undo rebuilds the same command from its stored snapshot



Sequence diagram · undo the selection (FR-04.3)

If more than 10 minutes have passed since `OfferSelectedAt`, `OfferSelectedState.UndoSelection` throws `UNDO_WINDOW_EXPIRED` (409) and nothing is saved.

9. Implementation Notes

The code lives in `IndustryApp/src`; every module (Reports, Offers, Contacts, Notifications, Accounts) has a folder with the same name in each layer.

Where UC-04 is implemented

Part	Location	Release	Author
State machine (`IReportState`, `ReportStateBase`, 5 states, `ReportStateFactory`)	`IndustryApp.Domain/Reports/States/`	v0.1	Ömer Efe Öztürk

Part	Location	Release	Author
`AccidentReport.SelectOffer`, `UndoSelection`, `IsMatchedWith`; `ReportCommandSnapshot`	`IndustryApp.Domain/Reports/`	v0.1	Ömer Efe Öztürk
`ContactInfo`, `ContactInfoProxy`, `IContactInfo`	`IndustryApp.Domain/Contacts/`	v0.1	Ömer Efe Öztürk
Schema, partial unique index, xmin token, `UnitOfWork`, `InProcessEventDispatcher`	`Infrastructure/Persistence/`, `Infrastructure/Events/`	v0.1	Ömer Efe Öztürk
Domain unit tests (State, Builder, Proxy, price) and frontend skeleton	`tests/IndustryApp.UnitTests/Domain/`, `src/frontend/industryapp-web/`	v0.1	Bahadır İhsan Herdem
`IReportCommand`, `SelectOfferCommand`, `IReportCommandFactory`, `ReportCommandFactory`, `ReportCommandNames`	`IndustryApp.Application/Offers/Commands/`	v0.2	Ömer Efe Öztürk
`OfferSelectionService` and selection contracts	`IndustryApp.Application/Selection/`	v0.2	Ömer Efe Öztürk
`ContactService`, `MatchContactResponse`	`IndustryApp.Application/Contacts/`	v0.2	Ömer Efe Öztürk
`ReportService` (detail, complete, cancel), `ReportDetailResponse`	`IndustryApp.Application/Reports/`	v0.2	Ömer Efe Öztürk
`OfferSelectionController`, `ContactsController`, `ReportsController`	`IndustryApp.WebApi/Controllers/`	v0.2	Ömer Efe Öztürk
UC-04 local test data	`database/dev/uc04-dev-data.sql`	v0.2	Ömer Efe Öztürk
UC-04 screens: `CustomerReportDetailView`, `OfferList`, `SortSelect`, `PriceBreakdown`, `ConfirmSelectDialog`, `SelectionBanner`, `MatchView`	`src/frontend/industryapp-web/src/modules/{reports,offers,selection,contacts}/`	v0.2	Ömer Efe Öztürk
UC-04 unit and integration tests	`tests/IndustryApp.UnitTests/Application/`, `tests/IndustryApp.IntegrationTests/`	v0.2	Ömer Efe Öztürk

Part	Location	Release	Author
Sort strategies, `OfferService`, `OffersController`	`IndustryApp.Application/Offers/Sorting/`, `WebApi/Controllers/`	v0.2	Bahadır İhsan Herdem
Identity, JWT, `CurrentUser`, `IdentityService`, registration factories, `AuthController`	`Infrastructure/Identity/`, `Application/Accounts/`, `WebApi/`	v0.2	Bahadır İhsan Herdem
`NotificationHub`, `SignalRRealtimePublisher`, UC-04 notification handlers, `NotificationsController`	`WebApi/Realtime/`, `Application/Notifications/`	v0.2	Bahadır İhsan Herdem
Frontend shared layer (http, auth, format, realtime), router, login/register screens, `NotificationBell`	`src/frontend/industryapp-web/src/shared/`, `src/app/`	v0.2	Bahadır İhsan Herdem

Coding rules applied

- Each pattern class starts with the comment template: `// [UC-04][PAT-04.n] <Pattern>` followed by *Intent*, *Reason*, *Benefit* and *Metric* lines with the verifying test.
- Entities have private constructors and static `Create(...)` methods; invalid objects cannot be built.
- Services return `Result<T>` instead of throwing for expected errors; `ApiControllerBase.ToActionResult` maps them to HTTP codes.
- Every service and repository sits behind an `I`-prefixed interface; requests are interfaces too (`ISelectOfferRequest`).
- Build runs with `TreatWarningsAsErrors`; no constant is hard-coded, limits come from `IOptions<T>`.
- Branch `uc-04/<short-name>`, commit `[UC-04] feat|fix|test|docs|refactor: <description>`; each PR is reviewed by the other member.

Undo across HTTP requests

HTTP is stateless, so the command object no longer exists when the undo request arrives. The command therefore persists itself: `CreateSnapshot()` stores the chosen and not-selected offer ids in `report_status_history.snapshot`. On undo, `ReportCommandFactory.Restore(...)` rebuilds the same command from the latest `SelectOfferCommand` row and calls `Undo()`.

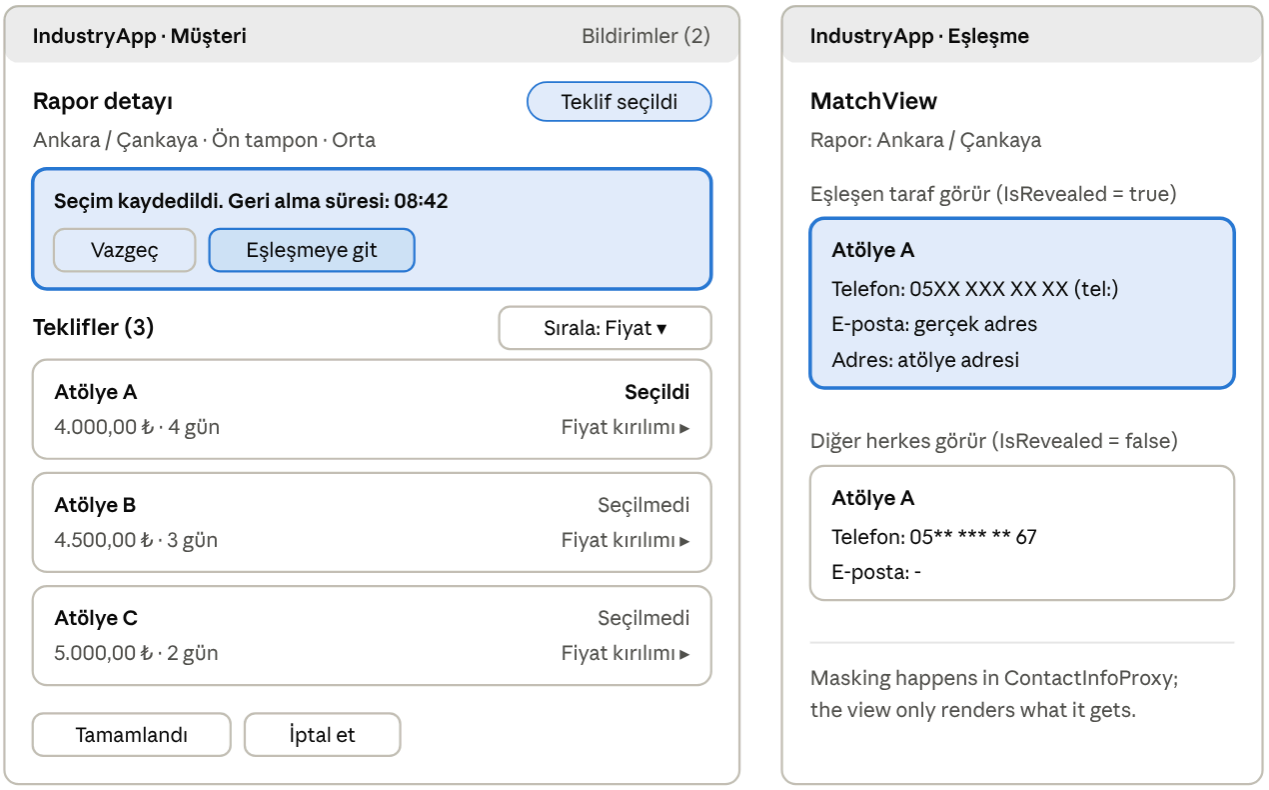
10. User Interface Design

UC-04 uses two screens and one layout component; all text is Turkish, enum values are shown as Turkish labels and empty values as `-`.

View	Route	Role	Content
`CustomerReportDetailView`	`/customer/reports/:reportId`	Customer	Report summary, status chip, `SelectionBanner`, `OfferList` with `SortSelect` and `PriceBreakdown`, confirm dialog, complete and cancel buttons
`MatchView`	`/match/:reportId`	Customer or matched mechanic	Other party's name, phone (`tel:` link), e-mail, workshop address; masked when `isRevealed` is false
`NotificationBell`	layout	Any signed-in user	Unread count, last 10 notifications, click opens the report

10.1 UI mockups / wireframes

The banner shows the undo countdown; contact opens only for the matched pair



Wireframes · CustomerReportDetailView with SelectionBanner, and MatchView

The offer amounts match the local test data script (`uc04-dev-data.sql`). The implemented screens follow this layout with MUI components.

10.2 Interaction rules

- "Bu teklifi seç" opens a confirm dialog; the button is disabled while the request runs.
- `SelectionBanner` counts down to `undoDeadline`; when it reaches zero the "Vazgeç" button disappears.
- "Tamamlandı" and "İptal et" appear only in states that allow them; the server still checks through State.
- `offer.*` and `report.statusChanged` SignalR events update the open screen without reload.
- API errors are shown as Turkish messages chosen by the ProblemDetails `code`.

11. External Interfaces

11.1 REST API (UC-04)

All endpoints require a valid JWT; JSON is camelCase. Numbers in the first column match the project-wide endpoint list.

#	Method and path	Policy	Service call	FR
18	GET /api/reports/{reportId}	CustomerOnly	`IReportService.GetForCustomerAsync`	FR-04.1
19	POST /api/reports/{reportId}/cancel	CustomerOnly	`IReportService.CancelAsync`	FR-04.5
20	POST /api/reports/{reportId}/complete	CustomerOnly	`IReportService.CompleteAsync`	FR-04.5
27	GET /api/reports/{reportId}/offers?sort=price|fastest|newest	CustomerOnly	`IOfferService.ListForReportAsync`	FR-04.1
29	POST /api/reports/{reportId}/selection	CustomerOnly	`IOfferSelectionService.SelectAsync`	FR-04.2
30	DELETE /api/reports/{reportId}/selection	CustomerOnly	`IOfferSelectionService.UndoAsync`	FR-04.3
31	GET /api/reports/{reportId}/selection	Authenticated	`IOfferSelectionService.GetAsync`	FR-04.2
32	GET /api/reports/{reportId}/contact	Authenticated	`IContactService.GetMatchContactAsync`	FR-04.4
33–36	GET/POST /api/notifications…	Authenticated	`INotificationService`	all

Example – select an offer:

```
POST /api/reports/3f2a…/selection
{ "offerId": "9c21…" }

200 OK
{ "reportId": "3f2a…", "offerId": "9c21…",
  "selectedAt": "2026-11-06T10:15:00Z", "undoDeadline": "2026-11-
06T10:25:00Z" }
```

Errors use RFC 7807 ProblemDetails with a machine-readable `code` (section 13).

11.2 Real-time interface (SignalR)

One hub, `/hubs/notifications`, pushes server-to-client messages only; every action goes through REST. The JWT is read from the `access_token` query parameter on `/hubs` paths.

Event	Receiver	Payload	Client effect
`notification.created`	Affected user	id, type, title, body, payload, createdAt	Pop-up, unread count +1

Event	Receiver	Payload	Client effect
`offer.created` / `offer.updated` / `offer.withdrawn`	Report owner	reportId, offer data	Offer list updates live
`report.statusChanged`	Customer and bidding mechanics	reportId, status, selectedOfferId	Screen reloads, buttons follow the status

11.3 Third-party systems

System	Used for	Access
PostgreSQL 16	All persistent data	EF Core + Npgsql, behind repositories
Browser (React SPA)	UI, SignalR client	REST + WebSocket via Vite proxy in development
SMTP provider	E-mail channel	Not used by UC-04 in v0.2; e-mail deliveries are recorded as `Skipped` until v0.3

12. Performance Considerations

12.1 Performance requirements

Code	Requirement	How UC-04 meets it
NFR-P1	API responses under 500 ms at p95 under normal load	Selection is one transaction on one report; notifications run after the response
PR-04.1	A new offer or status change reaches the open screen in under 3 s	SignalR push instead of polling
PR-04.2	Offer list query reads only needed columns	`AsNoTracking()` + projection to `OfferListItemResponse`
PR-04.3	Sorting never loads all rows into memory	`IOfferSortStrategy.Apply(IQueryable<Offer>)` adds `ORDER BY` to the SQL

12.2 Scalability and optimization strategies

- **Asynchronous side effects.** Domain events are queued after commit; `EventProcessingBackgroundService` sends notifications, so the HTTP response never waits for them (PAT-03.1).
- **Indexes for UC-04 queries.** `ix_offers_report_status (report_id, status)` for the offer list, `ix_report_status_history_report (report_id, occurred_at DESC)` for undo, `ix_notifications_user_unread` for the bell.
- **Concurrency without locks.** Optimistic concurrency (PostgreSQL `xmin`) plus the partial unique index `ux_offers_one_accepted_per_report` stop double selection without holding row locks.
- **Separate aggregates.** `Offer` is its own aggregate root, so mechanics writing offers never lock the whole report.
- **Client caching.** TanStack Query caches report and offer queries; SignalR events update the cache instead of refetching everything.
- **Scale-out path (future).** Multiple API instances need a Redis backplane for SignalR, an advisory lock for background jobs and a transactional outbox for events (section 17).

13. Error Handling and Logging

13.1 Exception management

Expected errors travel as values, unexpected ones as exceptions; both end as RFC 7807 ProblemDetails with a `code` field that the frontend maps to a Turkish message.

Source	Mechanism	HTTP	UC-04 codes
Ownership / relation check in a service	`Error.Forbidden` in `Result`	403	`NOT_REPORT_OWNER`, `NOT_MATCHED`
Missing entity	`Error.NotFound`	404	`REPORT_NOT_FOUND`, `OFFER_NOT_FOUND`, `CONTACT_NOT_FOUND`
Empty `offerId`	`Error.Validation`	400	`VALIDATION_FAILED`

Source	Mechanism	HTTP	UC-04 codes
Offer no longer Pending	`Error.Conflict` in `OfferSelectionService`, `DomainException` in `AccidentReport.SelectOffer`	409	`OFFER_NOT_AVAILABLE`
Forbidden state transition	`DomainException` from `ReportStateBase.Deny`	409	`REPORT_INVALID_STATE`
Undo after the window	`DomainException` from `OfferSelectedState`	409	`UNDO_WINDOW_EXPIRED`
Undo or contact without an active selection	`Error.Conflict`	409	`REPORT_INVALID_STATE`, `NO_ACTIVE_SELECTION`
Concurrent selection	`UnitOfWork`: xmin mismatch / PostgreSQL 23505 → `ConflictException`	409	`CONCURRENCY_CONFLICT`, `UNIQUE_VIOLATION`
Anything else	`GlobalExceptionHandler` (.NET 8 `IExceptionHandler`)	500	`UNEXPECTED_ERROR`; no internal details, `traceId` returned

```
{ "type": "https://industryapp.com/errors/undo-window-expired",
  "title": "Seçimi geri alma süresi doldu.", "status": 409,
  "code": "UNDO_WINDOW_EXPIRED", "traceId": "00-4bf92f35…" }
```

In the background queue a failing handler is logged and skipped; it does not stop other handlers or the queue.

13.2 Logging mechanisms

- **Serilog** writes to the console and to `logs/api-YYYYMMDD.log` (daily file); minimum level Information, Microsoft at Warning.
- **Correlation id.** `CorrelationIdMiddleware` adds `X-Correlation-Id` to every response and to every log line of that request.
- **Audit trail.** Every selection and undo writes a `report_status_history` row: from/to status, command name, actor, snapshot, time.
- **Delivery log.** `notification_deliveries` records each channel result (`Sent`, `Failed`, `Skipped`, `CircuitOpen`).
- **Privacy.** Phone numbers and e-mail addresses are never written to logs.

14. Design for Testability

Every UC-04 rule can be tested without a database, a web server or a real clock.

- **Interfaces everywhere.** Services depend on `IAccidentReportRepository`, `IOfferRepository`, `IUnitOfWork`, `ICurrentUser`, `IEventDispatcher`; tests replace them with NSubstitute fakes.
- **Controllable time.** The domain takes `now` as a parameter and services use `IDateTimeProvider`; the 10-minute window is tested in milliseconds.
- **Pure domain.** State, Command and Proxy classes have no infrastructure dependency, so they run in plain xUnit tests.
- **Real database where it matters.** Concurrency and the partial unique index are tested against PostgreSQL in Testcontainers; each test class runs the migration and `reference-data.sql`.

Tests for UC-04

Code	Level	Test method	Verifies
TC-04.1.1	Unit	`SelectOffer_NotOwner_ReturnsForbidden`	Alt. flow 2a
TC-04.1.2–4	Unit	One test per sort strategy	PAT-04.4, FR-04.1
TC-04.2.1	Unit	`OpenState_SelectOffer_MovesToOfferSelected`	PAT-04.1, FR-04.2
TC-04.2.2	Unit	`CompletedState_SelectOffer_Throws`	PAT-04.1
TC-04.2.3	Integration	`ConcurrentSelect_SecondRequestGets409`	xmin + partial unique index, alt. flow 3a
TC-04.3.1	Unit	`Undo_WithinWindow_RestoresPendingOffers`	PAT-04.2, FR-04.3
TC-04.3.2	Unit	`Undo_AfterWindow_Throws`	FR-04.3, alt. flow 6b
TC-04.4.1	Unit	`ContactProxy_NotMatched_ReturnsMaskedPhone`	PAT-04.3, FR-04.4

Code	Level	Test method	Verifies
TC-04.4.2	Integration	`Contact_NotMatchedMechanic_Gets403OrMasked`	FR-04.4, alt. flow 5a (E2E-07)
–	Unit	`SelectOffer_Owner_SavesSnapshotAndReturnsUndoDeadline`, `SelectThenUndo_WithinWindow_ReopensAllOffers`, `Undo_WithoutSelection_ReturnsConflict`, `SelectOffer_WithdrawnOffer_ReturnsOfferNotAvailable`	FR-04.2, FR-04.3
–	Unit	`Contact_SelectedMechanic_SeesRealCustomerPhone`, `Contact_Customer_SeesWinningMechanic`, `Contact_NotSelectedMechanic_GetsMaskedPhone`, `Contact_UnrelatedUser_ReturnsForbidden`	PAT-04.3, FR-04.4
–	Unit	`Cancel_OpenReport_ClosesPendingOffers`, `Complete_OpenReport_IsRejectedByState`, `Complete_NotOwner_ReturnsForbidden`	FR-04.5
–	Integration	`SelectThenUndo_PersistsSnapshotAndRestoresOffers`	PAT-04.2 snapshot in jsonb

Integration tests run the real services, repositories and `UnitOfWork` against PostgreSQL 16 in Testcontainers (migration + `reference-data.sql`); only `ICurrentUser` and `IIdentityService` are test doubles. All unit and integration tests pass in release v0.2 (dotnet test).

Manual end-to-end scenarios

Code	Scenario	Expected result
E2E-04	Customer selects an offer	Winner gets "you were selected", others "not selected"; match screen shows numbers
E2E-05	Customer undoes within 10 minutes	Report Open, offers Pending; mechanics notified
E2E-07	Unmatched mechanic calls the contact endpoint	403 or masked number

15. Deployment and Installation Design

In v0.2 IndustryApp runs in a development environment: only PostgreSQL is in Docker, the API and the frontend run on the host. Production packaging (API and web containers behind Caddy) is delivered in v0.4.

15.1 Environment configuration

Environment	API and frontend	Database	Secrets
Development (v0.2)	`dotnet run` on :5080, `npm run dev` on :5173 (Vite proxies `/api`, `/hubs`)	`docker compose up -d db`	`dotnet user-secrets`
Production (v0.4)	`docker compose up -d --build` (api, web, Caddy)	`pgdata` volume	Environment variables, e.g. `Jwt__SigningKey`

Setting	Default	Used by
`Offers:SelectionUndoWindowMinutes`	10	`OfferSelectionService`, `OfferSelectedState`
`Jwt:AccessTokenMinutes` / `RefreshTokenDays`	30 / 14	`JwtTokenService`
`ConnectionStrings:Default`	user-secrets	`AppDbContext`
`Jwt:SigningKey`	user-secrets	JWT signing and validation

15.2 Installation steps (development)

1. `docker compose up -d db` (set `DB_PORT` in `.env` if 5432 is taken).
2. Set `ConnectionStrings:Default` and `Jwt:SigningKey` with `dotnet user-secrets`.
3. `dotnet tool restore` and `dotnet ef database update -p IndustryApp.Infrastructure -s IndustryApp.WebApi`.
4. Load reference data: `psql … < database/reference-data.sql`.
5. `dotnet build`, `dotnet test`, `dotnet run --project IndustryApp.WebApi` (Swagger at `/swagger`, health at `/health`).
6. `npm install` and `npm run dev` in `src/frontend/industryapp-web`.

7. Register one customer and three mechanics on screen, approve the mechanics with SQL, then run `database/dev/uc04-dev-data.sql` to create a report with offers.

15.3 Packaging and dependencies

- Backend: one solution with 4 projects + 2 test projects; NuGet restore; `Directory.Build.props` sets net8.0, nullable and warnings-as-errors.
- Frontend: npm with a lock file; Vite builds static files for the web container.
- Database: EF Core migrations; `database/initial.sql` is the SQL script of the initial migration for review. Production runs a migrations bundle, never migrates at start-up.

16. Change Log

Version	Date	Author	Change
0.2	09/11/2026	Ömer Efe Öztürk, Bahadır İhsan Herdem	UC-04 end to end: Command, sort strategies, selection/undo/contact services, controllers, SignalR, UC-04 screens, tests; identity and registration (UC-01, PAT-01.1); this SDD
0.1	21/10/2026	Ömer Efe Öztürk	Phase 1 infrastructure: 6-project solution, domain model with State and Proxy skeletons, 21-table schema, reference data, event bus, Swagger, `/health`
0.0	30/09/2026	Bahadır İhsan Herdem	Repository created

17. Future Work / Open Issues

Item	Why not now	Effect when done
Report creation and offer submission (UC-02, UC-03)	Planned for v0.3	Dev data script is deleted; UC-04 runs on real reports
E-mail notifications for UC-04 events	SMTP channel arrives with UC-03	Winner and other mechanics also get e-mail
Role menus from Abstract Factory (PAT-01.2)	Planned for v0.4	Layouts read menus from `/api/account/dashboard`

Item	Why not now	Effect when done
Transactional outbox	Single server; in-memory queue is enough	No lost events if the process stops after commit
Multiple API instances	Traffic does not need it	Redis backplane for SignalR, advisory lock for jobs
Refresh token in httpOnly cookie	Simplicity in the first release	Better protection against XSS
"Sort by rating"	Rating is out of scope	One new `IOfferSortStrategy` class, no other change
In-app messaging, payment	Out of scope	New modules subscribe to `OfferSelectedEvent`

Open issue: after the undo window closes the customer can only cancel the report, not re-select; whether a second selection should be allowed is still open.